JavaScript Hoisting

Coding Practice Questions

58 output-prediction questions covering var, let, const, the Temporal Dead Zone, function declarations vs. expressions, closures, and class hoisting.

Includes full answers and explanations at the end of each section.

Contents

Section A: var Hoisting Basics (Q1–Q8)
Section B: let & const and the Temporal Dead Zone (Q9–Q18)
Section C: Function Declarations vs Function Expressions (Q19–Q28)
Section D: Hoisting Inside Functions & Blocks (Q29–Q38)
Section E: Loops, Closures & Hoisting (Q39–Q45)
Section F: Classes & Hoisting (Q46–Q51)
Section G: Tricky / Interview-Style Mixed Questions (Q52–Q58)

Questions

Section A: var Hoisting Basics

Q1. What is logged / what happens?

```
console.log(a);
var a = 5;
console.log(a);
```

Q2. What is logged / what happens?

```
console.log(x);
var x;
```

Q3. What is logged / what happens?

```
console.log(typeof y);
```

Q4. What is logged / what happens?

```
var a = 1;
function test() {
console.log(a);
var a = 2;
}
test();
```

Q5. What is logged / what happens?

```
console.log(b);
b = 10;
var b;
```

Q6. What is logged / what happens?

```
for (var i = 0; i < 3; i++) {}
console.log(i);
```

Q7. What is logged / what happens?

```
function foo() {
console.log(bar);
var bar = 'hoisted';
return bar;
}
foo();
```

Q8. What is logged / what happens?

```
var a = 1;
function outer() {
console.log(a);
var a = a + 1;
}
outer();
```

Section B: let & const and the Temporal Dead Zone

Q9. What is logged / what happens?

```
console.log(a);
let a = 5;
```

Q10. What is logged / what happens?

```
console.log(typeof a);
let a = 5;
```

Q11. What is logged / what happens?

```
{
console.log(b);
const b = 10;
}
```

Q12. What is logged / what happens?

```
let x = 1;
{
console.log(x);
let x = 2;
}
```

Q13. What is logged / what happens?

```
function test() {
console.log(a);
if (true) {
let a = 10;
}
}
test();
```

Q14. What is logged / what happens?

```
let x = 10;
function foo() {
console.log(x);
let x = 20;
}
foo();
```

Q15. What is logged / what happens?

```
const obj = { a: 1 };
obj.a = 2;
console.log(obj.a);
```

Q16. What is logged / what happens?

```
try {
console.log(a);
} catch (e) {
console.log(e.message);
}
let a = 1;
```

Q17. What is logged / what happens?

```
console.log(typeof notDeclared);
console.log(typeof inTDZ);
let inTDZ;
```

Q18. What is logged / what happens?

```
for (let i = 0; i < 3; i++) {}
console.log(i);
```

Section C: Function Declarations vs Function Expressions

Q19. What is logged / what happens?

```
foo();
function foo() {
console.log('hello');
}
```

Q20. What is logged / what happens?

```
bar();
var bar = function () {
console.log('hi');
};
```

Q21. What is logged / what happens?

```
baz();
const baz = function () {
console.log('hi');
};
```

Q22. What is logged / what happens?

```
console.log(typeof greet);
function greet() {}
```

Q23. What is logged / what happens?

```
foo();
function foo() {
console.log('first');
}
function foo() {
console.log('second');
}
```

Q24. What is logged / what happens?

```
var foo = 'value';
function foo() {}
console.log(typeof foo);
```

Q25. What is logged / what happens?

```
console.log(foo);
var foo = function bar() {};
console.log(typeof bar);
```

Q26. What is logged / what happens?

```
if (true) {
function inner() { return 'block'; }
}
console.log(typeof inner);
```

Q27. What is logged / what happens?

```
function outer() {
return inner();
function inner() {
return 'called';
}
}
console.log(outer());
```

Q28. What is logged / what happens?

```
var x = 1;
(function () {
console.log(x);
var x = 2;
})();
```

Section D: Hoisting Inside Functions & Blocks

Q29. What is logged / what happens?

```
function test() {
console.log(a);
console.log(b);
var a = 1;
let b = 2;
}
test();
```

Q30. What is logged / what happens?

```
function test() {
if (false) {
var a = 10;
}
console.log(a);
}
test();
```

Q31. What is logged / what happens?

```
function test() {
console.log(a);
{
var a = 5;
}
}
test();
```

Q32. What is logged / what happens?

```
function test() {
return;
var a = 1;
console.log(a);
}
console.log(test());
```

Q33. What is logged / what happens?

```
function test(a) {
console.log(a);
var a = 5;
console.log(a);
}
test(10);
```

Q34. What is logged / what happens?

```
function test() {
console.log(arguments);
var arguments = 'shadowed';
console.log(arguments);
}
test(1, 2);
```

Q35. What is logged / what happens?

```
console.log(this);
var this_test = 'value';
```

Q36. What is logged / what happens?

```
function test() {
var a = 1;
function a() {}
console.log(typeof a);
}
test();
```

Q37. What is logged / what happens?

```
function test() {
console.log(a);
var a;
a = 10;
var a;
console.log(a);
}
test();
```

Q38. What is logged / what happens?

```
function outer() {
var a = 'outer';
function inner() {
console.log(a);
var a = 'inner';
}
inner();
}
outer();
```

Section E: Loops, Closures & Hoisting

Q39. What is logged / what happens?

```
for (var i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 0);
}
```

Q40. What is logged / what happens?

```
for (let i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 0);
}
```

Q41. What is logged / what happens?

```
var funcs = [];
for (var i = 0; i < 3; i++) {
funcs.push(function () { return i; });
}
console.log(funcs.map(f => f()));
```

Q42. What is logged / what happens?

```
function makeCounters() {
var counters = [];
for (let i = 0; i < 3; i++) {
counters.push(() => i);
}
return counters;
}
console.log(makeCounters().map(f => f()));
```

Q43. What is logged / what happens?

```
(function () {
console.log(typeof value);
if (false) {
var value = 1;
}
})();
```

Q44. What is logged / what happens?

```
let count = 0;
function increment() {
count++;
let count = 10;
}
increment();
```

Q45. What is logged / what happens?

```
console.log(a, b, c);
var a = 1;
let b = 2;
const c = 3;
```

Section F: Classes & Hoisting

Q46. What is logged / what happens?

```
const p = new Person('Sam');
console.log(p.name);
class Person {
constructor(name) {
this.name = name;
}
}
```

Q47. What is logged / what happens?

```
console.log(typeof Animal);
class Animal {}
```

Q48. What is logged / what happens?

```
class Shape {
static create() {
return new Shape();
}
}
function test() {
console.log(typeof Shape);
class Shape {}
}
test();
```

Q49. What is logged / what happens?

```
class Base {
greet() {
return 'base';
}
}
class Derived extends Base {}
console.log(new Derived().greet());
```

Q50. What is logged / what happens?

```
{
console.log(typeof Vehicle);
let Vehicle = 5;
class Vehicle {}
}
```

Q51. What is logged / what happens?

```
function factory() {
return class Widget {
static label() { return Widget.name; }
};
}
const W = factory();
console.log(W.label());
```

Section G: Tricky / Interview-Style Mixed Questions

Q52. What is logged / what happens?

```
var a = 1;
function foo() {
console.log(a);
if (!a) {
var a = 2;
}
console.log(a);
}
foo();
```

Q53. What is logged / what happens?

```
console.log(a);
var a = function () {
console.log(b);
let b = 5;
};
a();
```

Q54. What is logged / what happens?

```
(function () {
console.log(typeof foo);
console.log(typeof bar);
var foo = 'foo';
function bar() {}
})();
```

Q55. What is logged / what happens?

```
let x = 'outer';
{
console.log(x);
x = 'reassigned';
let x = 'inner';
}
```

Q56. What is logged / what happens?

```
function test() {
var x = 1;
{
console.log(x);
let x = 2;
console.log(x);
}
console.log(x);
}
test();
```

Q57. What is logged / what happens?

```
console.log(foo());
function foo() {
return bar();
function bar() {
return 'nested hoist';
}
}
```

Q58. What is logged / what happens?

```
var x = 10;
function foo() {
x = 20;
return;
var x;
}
foo();
console.log(x);
```

Answers & Explanations

Section A: var Hoisting Basics

Q1 Answer

```
console.log(a);
var a = 5;
console.log(a);
```

Output

```
undefined
5
```

Explanation: The declaration 'var a' is hoisted to the top of its scope, but the assignment stays in place. So the first log sees 'a' declared but not yet assigned -> undefined.

Q2 Answer

```
console.log(x);
var x;
```

Output

```
undefined
```

Explanation: Only the declaration is hoisted. Since there is no assignment anywhere, x remains undefined.

Q3 Answer

```
console.log(typeof y);
```

Output

```
undefined
```

Explanation: 'y' is never declared, but typeof on an undeclared identifier does not throw - it returns the string 'undefined'.

Q4 Answer

```
var a = 1;
function test() {
console.log(a);
var a = 2;
}
test();
```

Output

```
undefined
```

Explanation: Inside test(), 'var a' is hoisted to the top of the function scope, shadowing the outer 'a'. So the local 'a' is undefined at the time of the log, not 1.

Q5. Answer

```
console.log(b);
b = 10;
var b;
```

Output

```
10
```

Explanation: 'var b' is hoisted (declaration only). The assignment 'b = 10' runs before the console.log that comes after it in source order? No - reorder: log runs first when b is still undeclared-but-hoisted -> undefined. (Trick: read the code order again: log, then b=10, then var b - log happens BEFORE b=10, so answer is undefined.)

Q6. Answer

```
for (var i = 0; i < 3; i++) {}
console.log(i);
```

Output

```
3
```

Explanation: 'var' is function/global scoped, not block scoped. After the loop finishes, i still exists in the enclosing scope with its final value, 3.

Q7. Answer

```
function foo() {
console.log(bar);
var bar = 'hoisted';
return bar;
}
foo();
```

Output

```
undefined
```

Explanation: 'var bar' declaration is hoisted to the top of foo(); its assignment happens after the console.log, so bar is undefined at that point.

Q8. Answer

```
var a = 1;
function outer() {
console.log(a);
var a = a + 1;
}
outer();
```

Output

```
undefined
```

Explanation: Because 'var a' is declared anywhere inside outer(), it is hoisted to the top of outer(), shadowing the global 'a' for the whole function. At console.log, local a is undefined; note 'a + 1' would also involve undefined + 1 = NaN, but that assignment happens after the log.

Section B: let & const and the Temporal Dead Zone

Q9. Answer

```
console.log(a);
let a = 5;
```

Output

```
ReferenceError: Cannot access 'a' before initialization
```

Explanation: 'let' declarations are hoisted to the top of the block, but they are not initialized until the actual declaration line runs. Accessing them before that point is in the Temporal Dead Zone (TDZ) and throws a ReferenceError.

Q10. Answer

```
console.log(typeof a);
let a = 5;
```

Output

```
ReferenceError: Cannot access 'a' before initialization
```

Explanation: Unlike an undeclared variable, 'a' IS declared (hoisted) here, so it's in the TDZ. Even 'typeof' throws for TDZ variables, unlike for a variable that was never declared at all.

Q11. Answer

```
{
console.log(b);
const b = 10;
}
```

Output

```
ReferenceError: Cannot access 'b' before initialization
```

Explanation: 'const' behaves like 'let' for hoisting: it is hoisted to the top of the block and stays in the TDZ until its declaration line executes.

Q12. Answer

```
let x = 1;
{
console.log(x);
let x = 2;
}
```

Output

```
ReferenceError: Cannot access 'x' before initialization
```

Explanation: The inner block has its own 'let x', which is hoisted to the top of that block, shadowing the outer x for the whole block. So the inner x is in the TDZ at the console.log, even though an outer x exists.

Q13. Answer

```
function test() {
console.log(a);
if (true) {
let a = 10;
}
}
test();
```

Output

```
ReferenceError: a is not defined
```

Explanation: Here 'a' is scoped only to the if-block, so outside of it, there is no hoisted 'a' at all in test()'s scope. This throws a plain 'not defined' error, not a TDZ error.

Q14. Answer

```
let x = 10;
function foo() {
console.log(x);
let x = 20;
}
foo();
```

Output

```
ReferenceError: Cannot access 'x' before initialization
```

Explanation: Even though an outer 'x' exists, the local 'let x' inside foo() is hoisted to the top of foo(), creating a TDZ for the entire function body before its declaration line.

Q15. Answer

```
const obj = { a: 1 };
obj.a = 2;
console.log(obj.a);
```

Output

```
2
```

Explanation: 'const' only prevents reassignment of the binding itself, not mutation of the object it points to. This is not a hoisting question per se, but a common companion trap - obj.a can be changed freely.

Q16. Answer

```
try {
console.log(a);
} catch (e) {
console.log(e.message);
}
let a = 1;
```

Output

```
Cannot access 'a' before initialization
```

Explanation: The TDZ error is a real, catchable ReferenceError, so it can be caught like any other error.

Q17 Answer

```
console.log(typeof notDeclared);
console.log(typeof inTDZ);
let inTDZ;
```

Output

```
undefined
ReferenceError: Cannot access 'inTDZ' before initialization
```

Explanation: 'notDeclared' was never declared anywhere, so typeof safely returns 'undefined'. 'inTDZ' IS declared later with let, so it's hoisted into TDZ, and typeof on it still throws.

Q18 Answer

```
for (let i = 0; i < 3; i++) {}
console.log(i);
```

Output

```
ReferenceError: i is not defined
```

Explanation: 'let' inside a for-loop header creates a new binding scoped to the loop (and even a fresh one per iteration). Once the loop ends, that binding does not leak into the outer scope.

Section C: Function Declarations vs Function Expressions

Q19 Answer

```
foo();
function foo() {
console.log('hello');
}
```

Output

```
hello
```

Explanation: Function declarations are fully hoisted - both their name AND their body/definition move to the top of the scope, so they can be called before their textual position.

Q20 Answer

```
bar();
var bar = function () {
console.log('hi');
};
```

Output

```
TypeError: bar is not a function
```

Explanation: Only the 'var bar' declaration is hoisted (as undefined); the function expression assignment happens later. At call time, bar is undefined, and calling undefined() throws a TypeError.

Q21 Answer

```
baz();
const baz = function () {
console.log('hi');
};
```

Output

```
ReferenceError: Cannot access 'baz' before initialization
```

Explanation: 'const baz' is hoisted into the TDZ (like all let/const bindings). Calling it before its declaration line triggers a TDZ ReferenceError, not a TypeError.

Q22 Answer

```
console.log(typeof greet);
function greet() {}
```

Output

```
function
```

Explanation: Function declarations hoist completely, including their implementation, so typeof correctly reports 'function' even before the textual declaration line.

Q23 Answer

```
foo();
function foo() {
console.log('first');
}
function foo() {
console.log('second');
}
```

Output

```
second
```

Explanation: When two function declarations share the same name, the later one overwrites the earlier one during hoisting, since both hoist to the top and the second assignment wins.

Q24 Answer

```
var foo = 'value';
function foo() {}
console.log(typeof foo);
```

Output

```
string
```

Explanation: Function declarations hoist first (with their full definition), then var declarations are processed but do NOT overwrite an existing hoisted value with undefined. Finally, the assignment foo = 'value' runs in source order, overwriting the function - so foo ends up as a string.

Q25. Answer

```
console.log(foo);
var foo = function bar() {};
console.log(typeof bar);
```

Output

```
undefined
ReferenceError: bar is not defined
```

Explanation: A named function expression's internal name ('bar') is only accessible inside the function's own body, not in the enclosing scope. Only 'foo' (the var) is hoisted to the outer scope.

Q26. Answer

```
if (true) {
function inner() { return 'block'; }
}
console.log(typeof inner);
```

Output

```
function
```

Explanation: This is implementation-defined by spec 'Annex B' legacy behavior: in non-strict mode, a function declared inside a block is also hoisted (as var-like) to the enclosing function/global scope, typically ending up defined outside the block too.

Q27. Answer

```
function outer() {
return inner();
function inner() {
return 'called';
}
}
console.log(outer());
```

Output

```
called
```

Explanation: Function declarations are hoisted to the top of their containing function, so 'inner' is fully defined before the 'return inner()' line executes, even though it's declared after the return statement in source.

Q28. Answer

```
var x = 1;
(function () {
console.log(x);
var x = 2;
})();
```

Output

```
undefined
```

Explanation: Inside the IIFE, 'var x' is hoisted to the top of that function scope, shadowing the outer x. The log therefore prints undefined rather than the outer value 1.

Section D: Hoisting Inside Functions & Blocks

Q29. Answer

```
function test() {
console.log(a);
console.log(b);
var a = 1;
let b = 2;
}
test();
```

Output

```
undefined
ReferenceError: Cannot access 'b' before initialization
```

Explanation: 'var a' hoists with an undefined value, so the first log succeeds. 'let b' hoists into the TDZ, so the second log throws before it can even print.

Q30. Answer

```
function test() {
if (false) {
var a = 10;
}
console.log(a);
}
test();
```

Output

```
undefined
```

Explanation: 'var' ignores block boundaries entirely - only the declaration (not the assignment) is hoisted to the top of the function, regardless of whether the if-block ever executes.

Q31. Answer

```
function test() {
console.log(a);
{
var a = 5;
}
}
test();
```

Output

```
undefined
```

Explanation: 'var a' inside the nested block is still hoisted to the top of the enclosing function (not the block), so 'a' exists as undefined at the first console.log.

Q32. Answer

```
function test() {
return;
var a = 1;
console.log(a);
}
console.log(test());
```

Output

```
undefined
```

Explanation: The function returns immediately, so 'console.log(a)' inside never runs, and test() itself returns undefined (no explicit return value). This tests that hoisting doesn't bypass control flow - the declaration hoists but the return still exits early.

Q33. Answer

```
function test(a) {
console.log(a);
var a = 5;
console.log(a);
}
test(10);
```

Output

```
10
5
```

Explanation: When a parameter shares a name with a 'var' declaration, the parameter's value initializes the binding. The 'var a' does not reset it to undefined - it just reuses the same binding, which is then reassigned to 5 by the explicit assignment.

Q34. Answer

```
function test() {
console.log(arguments);
var arguments = 'shadowed';
console.log(arguments);
}
test(1, 2);
```

Output

```
[Arguments] { '0': 1, '1': 2 }
shadowed
```

Explanation: 'arguments' is an implicit binding available in every regular function. A 'var arguments' declaration does not create a new hoisted-undefined slot that wipes it out early; it simply reassigns the existing arguments object when the assignment line executes.

Q35. Answer

```
console.log(this);
var this_test = 'value';
```

Output

```
the global object (or undefined in strict mode/modules)
```

Explanation: 'this' is not a hoistable variable - it's a special keyword bound at function-call time. It is unrelated to var/let/const hoisting; this question tests recognizing what is NOT affected by hoisting rules.

Q36. Answer

```
function test() {
var a = 1;
function a() {}
console.log(typeof a);
}
test();
```

Output

```
number
```

Explanation: Function declarations hoist before var declarations, giving 'a' the function initially. But then 'var a = 1' runs as an assignment in source order (var itself doesn't re-hoist over the function), overwriting it with the number 1.

Q37. Answer

```
function test() {
console.log(a);
var a;
a = 10;
var a;
console.log(a);
}
test();
```

Output

```
undefined
10
```

Explanation: Redeclaring the same variable with 'var' multiple times has no additional effect - it's still one binding. The first log happens before any assignment, then a becomes 10.

Q38. Answer

```
function outer() {
var a = 'outer';
function inner() {
console.log(a);
var a = 'inner';
}
inner();
}
outer();
```

Output

```
undefined
```

Explanation: Each function has its own hoisting scope. Inside inner(), 'var a' is hoisted to the top of inner(), shadowing outer's 'a' for the entire inner function body, so the log prints undefined, not 'outer'.

Section E: Loops, Closures & Hoisting

Q39. Answer

```
for (var i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 0);
}
```

Output

```
3
3
3
```

Explanation: 'var' creates a single shared binding for the whole loop. By the time the callbacks run (after the loop finishes), i has already reached 3, so all three callbacks log the same final value.

Q40. Answer

```
for (let i = 0; i < 3; i++) {
setTimeout(() => console.log(i), 0);
}
```

Output

```
0
1
2
```

Explanation: 'let' creates a brand-new binding scoped to each loop iteration. Each callback closes over its own iteration's 'i', so they print 0, 1, and 2 respectively.

Q41. Answer

```
var funcs = [];
for (var i = 0; i < 3; i++) {
funcs.push(function () { return i; });
}
console.log(funcs.map(f => f()));
```

Output

```
[3, 3, 3]
```

Explanation: All three closures share the single hoisted 'var i' binding, which equals 3 once the loop completes, so every function returns 3 when later invoked.

Q42. Answer

```
function makeCounters() {
var counters = [];
for (let i = 0; i < 3; i++) {
counters.push(() => i);
}
return counters;
}
console.log(makeCounters().map(f => f()));
```

Output

```
[0, 1, 2]
```

Explanation: Because 'let' gives each loop iteration its own fresh binding of i, each closure captures a different value, producing the expected sequence.

Q43. Answer

```
(function () {
console.log(typeof value);
if (false) {
var value = 1;
}
})();
```

Output

```
undefined
```

Explanation: Even though the if-block never executes, 'var value' is still hoisted to the top of the IIFE due to var's function-scoped, unconditional hoisting behavior.

Q44. Answer

```
let count = 0;
function increment() {
count++;
let count = 10;
}
increment();
```

Output

```
ReferenceError: Cannot access 'count' before initialization
```

Explanation: Inside increment(), the local 'let count' is hoisted to the top of the function, shadowing the outer count for the whole function body. So 'count++' touches the local TDZ binding and throws.

Q45. Answer

```
console.log(a, b, c);
var a = 1;
let b = 2;
const c = 3;
```

Output

```
ReferenceError: Cannot access 'b' before initialization
```

Explanation: JavaScript evaluates the console.log arguments left to right. 'a' would resolve to undefined fine, but evaluation stops the instant it reaches 'b', which is still in its TDZ, so a ReferenceError is thrown before anything is printed.

Section F: Classes & Hoisting

Q46. Answer

```
const p = new Person('Sam');
console.log(p.name);
class Person {
constructor(name) {
this.name = name;
}
}
```

Output

```
ReferenceError: Cannot access 'Person' before initialization
```

Explanation: Classes are hoisted like let/const - the binding exists but stays in the TDZ until the class definition is evaluated. Instantiating before that line throws.

Q47. Answer

```
console.log(typeof Animal);
class Animal {}
```

Output

```
ReferenceError: Cannot access 'Animal' before initialization
```

Explanation: Just like with let/const, even a 'typeof' check on a class identifier fails while it's in the TDZ - classes do not behave like function declarations here.

Q48. Answer

```
class Shape {
static create() {
return new Shape();
}
}
function test() {
console.log(typeof Shape);
class Shape {}
}
test();
```

Output

```
ReferenceError: Cannot access 'Shape' before initialization
```

Explanation: Inside test(), the local 'class Shape' declaration is hoisted to the top of the function (TDZ), shadowing the outer Shape class for the whole function body, so typeof on it still throws.

Q49. Answer

```
class Base {
greet() {
return 'base';
}
}
class Derived extends Base {}
console.log(new Derived().greet());
```

Output

```
base
```

Explanation: Not a hoisting trick, but worth confirming: Derived can be declared right after Base (in the same TDZ-respecting order) and inherits greet() normally through the prototype chain.

Q50. Answer

```
{
console.log(typeof Vehicle);
let Vehicle = 5;
class Vehicle {}
}
```

Output

```
SyntaxError: Identifier 'Vehicle' has already been declared
```

Explanation: You cannot declare both a 'let' variable and a 'class' with the same name in the same scope. This is a compile-time SyntaxError, thrown before any code in the block executes at all (so console.log never even runs).

Q51. Answer

```
function factory() {
return class Widget {
static label() { return Widget.name; }
};
}
const W = factory();
console.log(W.label());
```

Output

```
Widget
```

Explanation: Class expressions can still be named internally and hoisted-typed considerations don't block regular self-reference. This checks understanding that class name binding works inside its own static methods once the class is fully evaluated (not a TDZ trap here since it's used after definition).

Section G: Tricky / Interview-Style Mixed Questions

Q52. Answer

```
var a = 1;
function foo() {
console.log(a);
if (!a) {
var a = 2;
}
console.log(a);
}
foo();
```

Output

```
undefined
2
```

Explanation: 'var a' anywhere inside foo() hoists to the top, shadowing the outer a. First log: undefined. Since !undefined is truthy, the if-block runs, setting a to 2, so the second log is 2.

Q53. Answer

```
console.log(a);
var a = function () {
console.log(b);
let b = 5;
};
a();
```

Output

```
undefined
ReferenceError: Cannot access 'b' before initialization
```

Explanation: 'var a' is hoisted as undefined, and the console.log(a) on the first line runs before the assignment on the next line, so it prints undefined. By the time a() is actually called (after the assignment line has run), a is a real function, so the call succeeds and enters the function body - where the local 'let b' is in its TDZ, causing a ReferenceError.

Q54. Answer

```
(function () {
console.log(typeof foo);
console.log(typeof bar);
var foo = 'foo';
function bar() {}
})();
```

Output

```
undefined
function
```

Explanation: Function declarations hoist completely (so 'bar' is already callable/typed), while 'var foo' only hoists its declaration as undefined until its assignment line runs.

Q55. Answer

```
let x = 'outer';
{
console.log(x);
x = 'reassigned';
let x = 'inner';
}
```

Output

```
ReferenceError: Cannot access 'x' before initialization
```

Explanation: The block's own 'let x' is hoisted to the top of the block and shadows the outer x for the whole block, so both the console.log and the reassignment line refer to the TDZ binding, throwing on the first access.

Q56. Answer

```
function test() {
var x = 1;
{
console.log(x);
let x = 2;
console.log(x);
}
console.log(x);
}
test();
```

Output

```
ReferenceError: Cannot access 'x' before initialization
```

Explanation: The inner block declares its own 'let x', hoisted to the top of that block and shadowing the outer var x throughout the block. The very first console.log inside the block hits that TDZ and throws immediately.

Q57. Answer

```
console.log(foo());
function foo() {
return bar();
function bar() {
return 'nested hoist';
}
}
```

Output

```
nested hoist
```

Explanation: Nested function declarations hoist to the top of their own enclosing function. Inside foo(), 'bar' is fully defined before 'return bar()' executes, so the call succeeds and foo() returns 'nested hoist'.

Q58. Answer

```
var x = 10;
function foo() {
x = 20;
return;
var x;
}
foo();
console.log(x);
```

Outpu

```
10
```

Explanation: Inside foo(), 'var x' hoists a LOCAL x (undefined) to the top of the function, shadowing the outer x for the whole function body. So 'x = 20' sets the local x, not the global one. The global x printed afterward is unaffected, remaining 10.